

System Calls and Process Creation

Operating System Lab (5 Marks)

Name: Aniket Singh | Roll No: 2501012308 | Section: E

Environment: WSL (Linux) | Language: Python 3

Task-wise Summary

#	Task	System call / Command used
1	Create child process	os.fork()
2	Display PID and PPID	os.getpid(), os.getppid()
3	Wait for child	os.waitpid(pid, 0)
4	Run harmless Linux command	os.execvp('ls', ['ls', '-l'])
5	Create/write/read/close file	os.open(), os.write(), os.read(), os.close()
6	Inspect device / proc interface	/dev/null, /dev/zero, /proc/self/status
7	Handle invalid path / failed op	try/except FileNotFoundError, OSError
8	Show evidence	Summary output + screenshots below

Observations

- The parent (PID 538) created a child (PID 539); the child's PPID matched the parent's PID.
- The parent waited and received child exit status 0, so the command ran successfully.
- The child's image was replaced by **ls -l** using exec.
- File was created, written (31 bytes), read back and closed using low-level system calls.
- /dev/null discarded written data; /dev/zero returned null bytes; /proc/self/status exposed process info.
- An invalid path produced **[Errno 2] No such file or directory**, handled without crashing.

Execution Evidence

```
thean@MSI:/mnt/c/Users/thean$ mkdir -p ~/oslab && cd ~/oslab
cp /mnt/c/Users/thean/Downloads/syscalls_lab.py .
ls -l
total 4
-rwxr-xr-x 1 thean thean 2675 Oct  3 22:57 syscalls_lab.py
thean@MSI:~/oslab$ |
```

Screenshot 1: Setup - script copied into WSL working directory (~/oslab)

```
thean@MSI:~/oslab$ python3 syscalls_lab.py
=====
TASK 1: Create child process (os.fork)
=====

[PARENT] TASK 2: PID = 538, Child PID = 539

[PARENT] TASK 3: Waiting for child to complete...

[CHILD] TASK 2: Process IDs
[CHILD] PID = 539, PPID = 538

[CHILD] TASK 4: Executing harmless Linux command (ls -l)
total 4
-rwxr-xr-x 1 thean thean 2675 Oct  3 22:57 syscalls_lab.py
[PARENT] Child 539 finished, exit status = 0

TASK 5: File create/write/read/close
Read back: Hello from system call write()

TASK 6: Inspect /dev/null, /dev/zero, /proc
/dev/null -> write succeeded, data discarded
/dev/zero -> read 8 bytes: b'\x00\x00\x00\x00\x00\x00\x00\x00'
/proc/self/status -> Name:      python3
/proc/self/status -> Umask:    0022
/proc/self/status -> State:    R (running)

TASK 7: Error handling
Error: File not found -> [Errno 2] No such file or directory: '/invalid/path/file.txt'
Failed file operation: errno=2, message=No such file or directory

TASK 8: Evidence summary
Process   : parent PID 538, child PID 539
File      : test_file.txt (31 bytes)
Device    : /dev/null, /dev/zero, /proc/self/status accessed
Errors    : handled with try/except
thean@MSI:~/oslab$
```

Screenshot 2: Tasks 1-8 - process creation, PID/PPID, wait, exec, file I/O, device interfaces, error handling

```
thean@MSI:~/oslab$ ls -l /dev/null /dev/zero
head -5 /proc/self/status
ps -ef | head -5
crw-rw-rw- 1 root root 1, 3 Oct  3 22:47 /dev/null
crw-rw-rw- 1 root root 1, 5 Oct  3 22:47 /dev/zero
Name:      head
Umask:    0022
State:    R (running)
Tgid:     541
Ngid:     0
UID        PID    PPID  C STIME TTY          TIME CMD
root         1      0  0  22:46 ?        00:00:00 /sbin/init
root         2      1  0  22:46 ?        00:00:00 /init
root         6      2  0  22:46 ?        00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        42      1  0  22:46 ?        00:00:00 /usr/lib/systemd/systemd-journald
thean@MSI:~/oslab$ |
```

Screenshot 3: Task 6 & 8 - device files (/dev/null, /dev/zero), /proc/self/status and process list

Source Code: syscalls_lab.py

```
import os
import sys
import subprocess

TEST_FILE = "test_file.txt"

print("=" * 50)
print("TASK 1: Create child process (os.fork)")
print("=" * 50)

pid = os.fork()

if pid == 0:
    # ----- CHILD PROCESS -----
    print("\n[CHILD] TASK 2: Process IDs")
    print(f"[CHILD] PID = {os.getpid()}, PPID = {os.getppid()}")

    print("\n[CHILD] TASK 4: Executing harmless Linux command (ls -l)")
    sys.stdout.flush()
    os.execvp("ls", ["ls", "-l"]) # replaces child image
    # code below never runs if exec succeeds
else:
    # ----- PARENT PROCESS -----
    print(f"\n[PARENT] TASK 2: PID = {os.getpid()}, Child PID = {pid}")

    print("\n[PARENT] TASK 3: Waiting for child to complete...")
    child_pid, status = os.waitpid(pid, 0)
    print(f"[PARENT] Child {child_pid} finished, exit status = {os.WEXITSTATUS(status)}")

    # ----- TASK 5: File operations -----
    print("\nTASK 5: File create/write/read/close")
    fd = os.open(TEST_FILE, os.O_CREAT | os.O_WRONLY | os.O_TRUNC, 0o644)
    os.write(fd, b"Hello from system call write()\n")
    os.close(fd)

    fd = os.open(TEST_FILE, os.O_RDONLY)
    data = os.read(fd, 100)
    os.close(fd)
    print("Read back:", data.decode().strip())

    # ----- TASK 6: Device / proc interfaces -----
    print("\nTASK 6: Inspect /dev/null, /dev/zero, /proc")
    with open("/dev/null", "wb") as f:
        f.write(b"discarded data")
    print("/dev/null -> write succeeded, data discarded")

    with open("/dev/zero", "rb") as f:
        zeros = f.read(8)
    print("/dev/zero -> read 8 bytes:", zeros)

    with open("/proc/self/status") as f:
        for _ in range(3):
            print("/proc/self/status ->", f.readline().strip())

    # ----- TASK 7: Error handling -----
    print("\nTASK 7: Error handling")
    try:
        open("/invalid/path/file.txt", "r")
    except FileNotFoundError as e:
        print(f"Error: File not found -> {e}")
    except PermissionError as e:
        print(f"Error: Permission denied -> {e}")
    except OSError as e:
        print(f"Error: OS error [{e.errno}] -> {e.strerror}")

    try:
        os.open("/nonexistent_dir/x.txt", os.O_RDONLY)
    except OSError as e:
        print(f"Failed file operation: errno={e.errno}, message={e.strerror}")

    # ----- TASK 8: Evidence -----
    print("\nTASK 8: Evidence summary")
```

```
print(f"Process    : parent PID {os.getpid()}, child PID {pid}")
print(f"File       : {TEST_FILE} ({os.path.getsize(TEST_FILE)} bytes)")
print("Device      : /dev/null, /dev/zero, /proc/self/status accessed")
print("Errors       : handled with try/except")
os.remove(TEST_FILE)
```